LAPORAN PRAKTIKUM MODUL III

SISTEM PEMANTAUAN KUALITAS UDARA KOTA BANDUNG

Laporan Disusun Guna Memenuhi Tugas Mata Kuliah
GIK2CAB4 DASAR PEMROGRAMAN PERANGKAT BERGERAK



Oleh:

Kelas : D4SIKC-49-03

NIM	NAMA
707012500060	RICHIE RAHMATULLAH

PROGRAM STUDI S.Ter SISTEM INFORMASI KOTA CERDAS
FAKULTAS ILMU TERAPAN
UNIVERSITAS TELKOM
BANDUNG

2026

1.1 Tujuan Praktikum

Setelah mengikuti praktikum mahasiswa dapat mengenal dan menerapkan seperti

1. Memahami konsep dasar OOP (Object-Oriented-Programming) dalam bahasa dart
2. Mendeklarasikan class dalam Dart, yaitu class SensorUdara
3. Membuat object dari sebuah class dan menyimpannya dalam koleksi List
4. Menerapkan property (atribut) dan method (fungsi) beserta logika percabangan if-else di dalam class.
5. Menggunakan constructor dengan parameter bernama (named parameter) untuk memberi nilai awal pada objek

2.1 Alat dan Bahan

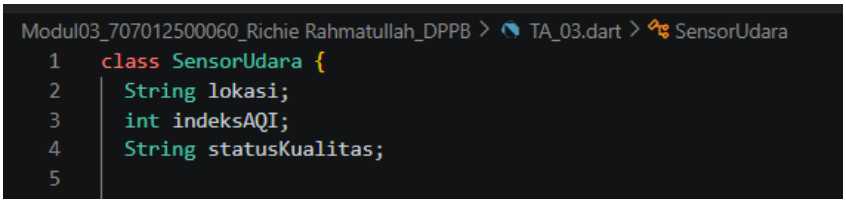
Alat dan bahan yang digunakan dalam praktikum ini adalah:

a. Perangkat Keras
    1. Laptop/Komputer dengan RAM minimal 4GB (disarankan 8GB), ruang penyimpanan kosong minimal 2GB, minimal CPU Intel core i3 Gen 8-11/ AMD Ryzen 3 3000 series (disarankan Intel Core i5 Gen 11+ dan AMD Ryzen 5 4000/5000 series)

b. Perangkat Lunak
    1. Visual Studio Code
    2. Brave Browser
    3. Flutter SDK

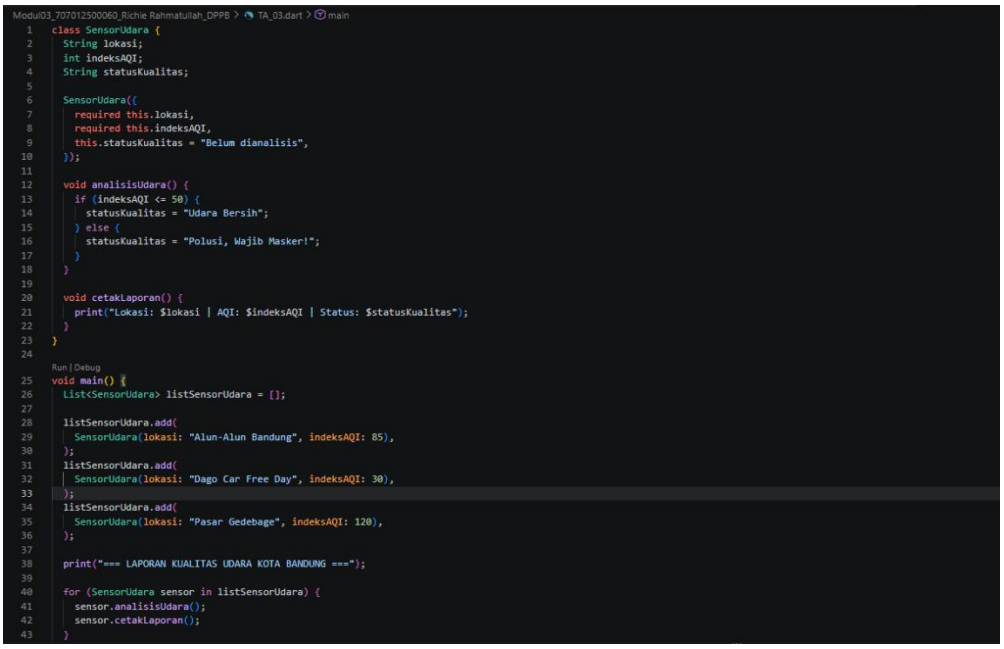
3.1 Langkah-Langkah Praktikum

1.1 Langkah umum: buat sebuah folder kerja, lalu buat file TA_03.dart di VS Code. Ketik kode program, jalankan dengan perintah dart run TA_03.dart, dan pastikan tidak ada error.



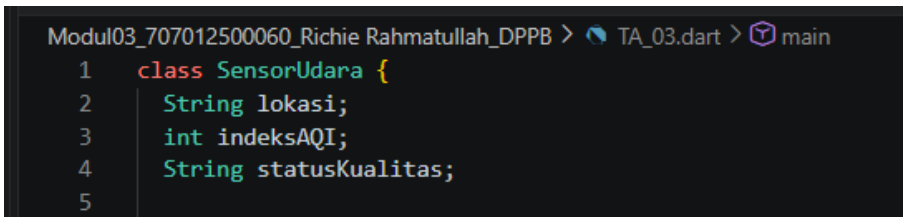
1. Kasus Tes Akhir: Sistem Pemantauan Kualitas Udara Kota Bandung (OOP)

   Pemerintah Kota Bandung baru saja memasang sensor kualitas udara di berbagai titik keramaian seperti Dago, Alun-Alun, dan Gedebage. Program ini mencatat data sensor, memproses status udaranya secara otomatis, dan menampilkan laporannya secara masal.



Penjelasan kode:

1. Program diawali dengan mendeklarasikan class SensorUdara sebagai cetak biru (blueprint) data sensor. Class ini memiliki tiga property: lokasi (String), indeksAQI (int), dan statusKualitas (String) yang diberi nilai awal bawaan "Belum dianalisis".



2. Constructor SensorUdara({required this.lokasi, required this.indeksAQI, this.statusKualitas = "Belum dianalisis"}) memakai kurung kurawal {} sehingga parameternya bersifat named parameter. Keyword required mewajibkan lokasi dan indeksAQI diisi saat objek dibuat, sedangkan statusKualitas boleh dikosongkan karena sudah punya nilai bawaan.

```
6   SensorUdara({
7     required this.lokasi,
8     required this.indeksAQI,
9     this.statusKualitas = "Belum dianalisis",
10  });
```

3. Method analisisUdara() tidak memiliki parameter. Method ini mengecek indeksAQI dengan if-else: jika nilainya kurang dari atau sama dengan 50, statusKualitas diubah menjadi "Udara Bersih"; selain itu diubah menjadi "Polusi, Wajib Masker!".

```
void analisisUdara() {
  if (indeksAQI <= 50) {
    statusKualitas = "Udara Bersih";
  } else {
    statusKualitas = "Polusi, Wajib Masker!";
  }
}
```

4. Method cetakLaporan() mencetak data objek ke layar dengan format Lokasi: ... | AQI: ... | Status: ... menggunakan string interpolation ($lokasi, $indeksAQI, $statusKualitas).

```
  void cetakLaporan() {
    print("Lokasi: $lokasi | AQI: $indeksAQI | Status: $statusKualitas");
  }
}
```

5. Pada fungsi main(), dibuat sebuah List<SensorUdara> bernama listSensorUdara. Tiga objek sensor baru, yaitu Alun-Alun Bandung (AQI 85), Dago Car Free Day (AQI 30), dan Pasar Gedebage (AQI 120), dimasukkan langsung di dalam .add() dengan metode Instance Creation Expression.

```dart
void main() {
  List<SensorUdara> listSensorUdara = [];

  listSensorUdara.add(
    SensorUdara(lokasi: "Alun-Alun Bandung", indeksAQI: 85),
  );
  listSensorUdara.add(
    SensorUdara(lokasi: "Dago Car Free Day", indeksAQI: 30),
  );
  listSensorUdara.add(
    SensorUdara(lokasi: "Pasar Gedebage", indeksAQI: 120),
  );

  print("=== LAPORAN KUALITAS UDARA KOTA BANDUNG ===");

  for (SensorUdara sensor in listSensorUdara) {
    sensor.analisisUdara();
    sensor.cetakLaporan();
  }
}
```

6. Terakhir, perulangan for-in memproses setiap objek di dalam List. Untuk tiap sensor, program memanggil analisisUdara() agar status ditentukan terlebih dahulu, lalu cetakLaporan() untuk menampilkan hasilnya.

```dart
  for (SensorUdara sensor in listSensorUdara) {
    sensor.analisisUdara();
    sensor.cetakLaporan();
  }
}
```

Hasil Output:

```
PS D:\Tugas Kuliah\SEMESTER 3 2026\DPPB\Latihan_02\Modul03_707012500060_Richie Rahmatullah_DPPB> dart TA_03.dart
=== LAPORAN KUALITAS UDARA KOTA BANDUNG ===
Lokasi: Alun-Alun Bandung | AQI: 85 | Status: Polusi, Wajib Masker!
Lokasi: Dago Car Free Day | AQI: 30 | Status: Udara Bersih
Lokasi: Pasar Gedebage | AQI: 120 | Status: Polusi, Wajib Masker!
PS D:\Tugas Kuliah\SEMESTER 3 2026\DPPB\Latihan_02\Modul03_707012500060_Richie Rahmatullah_DPPB>
```

Output membuktikan program berjalan sesuai rancangan. Ketiga objek SensorUdara berhasil dibuat dan diproses secara berurutan sesuai urutan penambahan ke dalam List. Dago Car Free Day dengan AQI 30 (di bawah batas 50) mendapat status "Udara Bersih", sedangkan Alun-Alun Bandung (AQI 85) dan Pasar Gedebage (AQI 120) yang berada di atas 50 mendapat status "Polusi, Wajib Masker!". Hasil ini sama persis dengan contoh output pada soal.

4.1 Analisis dan Pembahasan

Program TA_03.dart membahas konsep dasar Pemrograman Berorientasi Objek (OOP) pada Dart, yaitu class, object, property, constructor, dan method, yang digabungkan dalam satu alur logika backend sistem pemantauan kualitas udara Kota Bandung. Pada bagian class, SensorUdara dirancang sebagai cetak biru yang menyimpan tiga ciri sebuah sensor: lokasi pemasangan, nilai indeks AQI, dan status kualitas udara. Property statusKualitas diberi nilai bawaan "Belum dianalisis" karena status baru dapat ditentukan setelah data AQI diproses.

Pada bagian constructor, program memakai named parameter dengan keyword required. Pendekatan ini membuat pemanggilan seperti SensorUdara(lokasi: "Pasar Gedebage", indeksAQI: 120) lebih mudah dibaca dan urutan parameternya boleh dibolak-balik tanpa risiko data tertukar. Hal ini relevan dengan studi kasus, sebab kesalahan memasukkan lokasi ke kolom AQI akan langsung terdeteksi oleh compiler, dan data yang wajib diisi tidak bisa terlewat.

Pada bagian method, analisisUdara() menerapkan percabangan if-else dengan batas indeksAQI sebesar 50. Karena method ini berada di dalam class, ia dapat langsung membaca dan mengubah property milik objeknya sendiri tanpa parameter tambahan. Method cetakLaporan() memisahkan tugas menampilkan data dari tugas menganalisis data, sehingga setiap method hanya memiliki satu tanggung jawab dan mudah dipakai ulang.

Pada fungsi main(), objek-objek sensor dikumpulkan dalam List<SensorUdara> dan diproses dengan perulangan for-in. Dengan cara ini, program tetap sama meskipun jumlah sensor bertambah menjadi puluhan atau ratusan titik; programmer tidak perlu memanggil method satu per satu untuk setiap objek. Urutan pemanggilan di dalam perulangan juga penting: analisisUdara() harus dipanggil sebelum cetakLaporan(), karena jika dibalik, laporan akan menampilkan status awal "Belum dianalisis".

Dari sisi hasil, program menghasilkan dua kategori status sesuai ketentuan soal. Perlu dicatat bahwa pembagian dua kategori ini merupakan penyederhanaan. Pada penerapan sebenarnya, indeks kualitas udara biasanya dibagi ke dalam beberapa tingkat, sehingga method analisisUdara() dapat dikembangkan dengan else if untuk kategori yang lebih rinci.

Kendala yang umum terjadi saat mengerjakan tugas ini antara lain lupa menuliskan keyword required atau kurung kurawal pada constructor sehingga muncul error, lupa memanggil analisisUdara() sebelum cetakLaporan(), serta salah mengetik nama property saat string interpolation. Kendala ini dapat dihindari dengan membaca pesan error di terminal dan mencocokkannya kembali dengan ketentuan pada soal.

5.1 Kesimpulan

Dari Tes Akhir Praktikum yang telah dilaksanakan, dapat disimpulkan bahwa konsep dasar OOP pada Dart berhasil diterapkan dalam satu program yang utuh. Class SensorUdara berfungsi sebagai cetak biru, sedangkan objek-objek yang dibuat darinya mewakili sensor di lokasi yang berbeda dengan nilai property masing-masing. Constructor dengan named parameter dan keyword required memastikan data awal terisi lengkap dan terhindar dari kesalahan urutan. Method analisisUdara() dan cetakLaporan() memberikan perilaku pada objek, yaitu menentukan status udara secara otomatis dan menampilkan laporannya. Penggunaan List dan perulangan for-in memungkinkan seluruh objek diproses secara masal, berapa pun jumlahnya. Pemahaman ini menjadi dasar sebelum melanjutkan ke pengembangan aplikasi dengan Flutter.

6.1 Lampiran

https://github.com/RichieRahmatullah16/TA_Modul02_Richie-Rahmatullah_707012500060_DPPB.git